

# **Informe Tecnico Arquitectonico**

Arquitectura de Microservicios Serverless Resiliente  
para Gestion de Emergencias

Santiago Rodriguez

Universidad

Patrones Arquitectonicos Avanzados

Agosto 2026

## **Tabla de Contenido**

1. Vision General del Sistema
2. Patrones Arquitectonicos Aplicados
3. Stack Tecnologico
4. Microservicios
5. Flujo de un Reporte
6. Maquinas de Estado
7. Priorizacion Automatica
8. Frontend PWA
9. Base de Datos (Supabase + PostGIS)
10. Seguridad y Gestion de Secretos
11. Infraestructura AWS
12. Canary Deployment
13. CI/CD Pipeline
14. Gobernanza de Costos (AWS Budgets)
15. Diagramas C4
16. URLs de Produccion
17. Referencias

## 1. Vision General del Sistema

El sistema Emergencias MVP es una plataforma de gestion de emergencias post-sismo disenada para cuatro zonas de Colombia: Quibdo (Choco), Pereira, Cali y Manizales. Su objetivo es procesar, clasificar, enrutar y responder en tiempo real a las solicitudes de auxilio de la poblacion y los cuerpos de socorro (Cruz Roja, Bomberos, Defensa Civil y UNGRD).

El sistema tiene dos actores principales. El ciudadano reporta emergencias desde una Progressive Web App (PWA) con soporte offline, enviando tipo de emergencia, descripcion, foto y ubicacion GPS. El operador inicia sesion con credenciales de Supabase Auth, visualiza los reportes de su zona exclusivamente (gracias a Row-Level Security), asigna cuadrillas especializadas y monitorea el mapa interactivo con clustering DBSCAN.

## 2. Patrones Arquitectonicos Aplicados

**Microservicios.** Cuatro servicios independientes (S1 Intake, S2 Dispatch, S3 Geospatial, S4 Notification), cada uno con su propio Dockerfile, tests y dominio acotado. Cada servicio escala independientemente en Lambda.

**API Gateway.** nginx en local y AWS API Gateway HTTP API en produccion. Punto unico de entrada que rutea /reports a S1 y /dispatch a S2. Maneja CORS centralizadamente.

**Event-Driven Architecture.** publish\_event() emite eventos a EventBridge (AWS) o los loguea en local. Los servicios downstream reaccionan a eventos sin acoplamiento directo. S1 publica report.created y S3 y S4 reaccionan.

**State Machine.** Transiciones explicitas para reportes (recibido, despachado, en\_proceso, resuelto) y asignaciones (asignado, en\_ruta, en\_sitio, completado). Transiciones invalidas se rechazan con HTTP 409 Conflict. La logica esta en services/s2-dispatch/app/transitions.py.

**Strangler Fig (parcial).** El mock-organismo simula el sistema externo que se reemplazaria progresivamente. La interfaz del webhook ya esta definida; cuando exista el organismo real, solo se cambia la URL del endpoint sin tocar el codigo de S4.

**Database per Concern.** Una sola base de datos (Supabase/PostgreSQL) pero con acceso segmentado: los microservicios usan service\_role key (bypasea RLS) via PostgREST/RPC; los operadores usan anon key con RLS por zona.

**Offline-First (PWA).** IndexedDB como cola offline, service worker con VitePWA. Los reportes se encolan localmente y se sincronizan automaticamente cuando vuelve la conexion.

**Row-Level Security.** Politicas SQL que garantizan que cada operador solo acceda a datos de su zona. La seguridad se implementa a nivel de base de datos, no de aplicacion.

**Idempotency.** Cada reporte lleva un idempotency\_key (UUID v4) generado en el cliente, lo que evita duplicados si la PWA reintenta un envio offline.

3. Stack Tecnologico

| Capa             | Tecnologia                         | Justificacion                                     |
|------------------|------------------------------------|---------------------------------------------------|
| Frontend         | React 18 + Vite + TypeScript       | SPA ligera, tipado estatico                       |
| PWA              | VitePWA + IndexedDB                | Soporte offline nativo                            |
| Mapa             | Leaflet + react-leaflet            | Open-source, sin costo por tile                   |
| Backend          | Python 3.12 + FastAPI + Mangum     | Async, Pydantic, compatible con Lambda via Mangum |
| Base de datos    | Supabase (PostgreSQL 15 + PostGIS) | Auth, RLS, Realtime, funciones RPC                |
| Clustering       | PostGIS ST_ClusterDBSCAN           | DBSCAN en la base de datos via RPC                |
| Eventos          | AWS EventBridge                    | Bus de eventos serverless                         |
| Gateway          | nginx / AWS API Gateway            | Ruteo centralizado                                |
| Contenedores     | Docker + Docker Compose            | Desarrollo local reproducible                     |
| Hosting frontend | Vercel                             | Deploy automatico, CDN global                     |
| Hosting backend  | AWS Lambda + ECR                   | Serverless, escala a cero                         |
| Secretos         | AWS Secrets Manager                | Credenciales nunca en codigo                      |

Tabla 1. Stack tecnologico del sistema Emergencias MVP.

4. Microservicios

**S1 - Intake and Triage.** Recibe, valida y prioriza reportes nuevos. Valida campos con Pydantic, calcula prioridad automatica (P1-P4), asigna zona con zone\_lookup(lat, lon) RPC, guarda en Supabase y publica evento report.created.

**S2 - Dispatch and Resource Assignment.** Gestiona la asignacion de cuadrillas. Implementa la maquina de estados para asignaciones, crea `dispatch_assignments`, actualiza el status del reporte y rechaza transiciones invalidas con HTTP 409.

**S3 - Geospatial and Zone Aggregation.** Clustering geoespacial con DBSCAN. Llama a `refresh_zone_clusters()` RPC, donde PostGIS ejecuta `ST_ClusterDBSCAN`. Agrupa reportes cercanos en el mapa. Se dispara por evento, no por request del usuario.

**S4 - Notification and Status Broadcast.** Notifica organismos externos via webhook. En desarrollo usa mock-organismo (patron Strangler Fig). Se dispara por evento de EventBridge.

Cada microservicio usa Mangum como adaptador. Mangum traduce el evento de Lambda (JSON) al formato ASGI que FastAPI espera. Asi el mismo codigo funciona en local (uvicorn) y en produccion (Lambda) sin cambios.

## 5. Flujo de un Reporte

El flujo completo de un reporte sigue los siguientes pasos: (a) el ciudadano abre la PWA y llena el formulario con tipo de emergencia, descripcion, foto y ubicacion GPS; (b) el frontend genera un `idempotency_key` (UUID v4) y un `device_id`; (c) se envia POST `/reports` al API Gateway, que lo rutea a S1 Intake; (d) S1 valida los campos con Pydantic, calcula la prioridad automatica segun tipo y palabras clave, y asigna zona con `zone_lookup(lat, lon)`; (e) S1 inserta el reporte en Supabase con status recibido y publica el evento `report.created` a EventBridge; (f) S3 Geospatial reacciona al evento y recalcula clusters DBSCAN para la zona; (g) S4 Notification envia webhook al organismo externo; (h) el operador ve el reporte nuevo en su dashboard via Supabase Realtime, selecciona una cuadrilla y envia POST `/dispatch/assign`; (i) S2 crea la asignacion, actualiza el reporte a despachado y publica evento; (j) la cuadrilla avanza por los estados `en_ruta`, `en_sitio`, `completado`; (k) el reporte pasa a `en_proceso` y finalmente a `resuelto`; (l) el ciudadano consulta el estado en cualquier momento con su `device_id`.

## 6. Maquinas de Estado

**Estado del reporte:** recibido -> despachado -> en\_proceso -> resuelto. Cualquier estado puede pasar a descartado.

**Estado de la asignacion:** asignado -> en\_ruta -> en\_sitio -> completado. Cualquier estado puede pasar a cancelado.

Las transiciones invalidas se rechazan con HTTP 409 Conflict. La logica esta implementada en `services/s2-dispatch/app/transitions.py`, que define un diccionario de transiciones validas y verifica cada cambio de estado antes de ejecutarlo.

## **7. Priorizacion Automatica**

S1 asigna prioridad basada en el tipo de emergencia: Rescate recibe P1 (Critico), Medico recibe P2 (Urgente), Estructural recibe P3 (Moderado) y Preventivo recibe P4 (Preventivo).

Si la descripcion contiene palabras clave criticas como atrapado, colapso, no respira, inconsciente, sangrado, aplastado o derrumbe, la prioridad sube un nivel (P3 pasa a P2, P2 pasa a P1, P1 se mantiene). Esta logica esta implementada en `services/s1-intake/app/priority.py`.

## **8. Frontend PWA**

El frontend es una Progressive Web App construida con React 18, Vite y TypeScript, desplegada en Vercel. Cuenta con cuatro paginas principales: la vista de reporte del ciudadano (ruta /), la vista de seguimiento (/tracking), el login del operador (/operator/login) y el dashboard del operador (/operator).

Las capacidades offline incluyen: service worker registrado con VitePWA que precachea assets estaticos; cola offline en IndexedDB para guardar reportes sin conexion; hook `useOfflineSync` que sincroniza automaticamente cuando vuelve la conexion; hook `useOnlineStatus` que detecta cambios de conectividad; e `idempotency_key` para prevenir duplicados al reintentar. La aplicacion es instalable como app nativa en Android e iOS.

El mapa interactivo usa Leaflet con react-leaflet, mostrando reportes como marcadores geolocalizados y clusters DBSCAN agrupados por zona.

## **9. Base de Datos (Supabase + PostGIS)**

La persistencia usa Supabase (PostgreSQL 15 con la extension PostGIS habilitada). Las tablas principales son: `zones` (4 zonas con poligonos PostGIS), `reports` (reportes con tipo, prioridad, ubicacion geometry, status y zone\_id), `crews` (cuadrillas especializadas por zona), `dispatch_assignments` (registro de asignacion cuadrilla-reporte con estado) y `profiles` (perfil de operadores con zone\_id para RLS).

Se implementaron dos funciones RPC: `zone_lookup(lat, lon)`, que dado un punto devuelve en que zona cae usando `ST_Contains`; y `refresh_zone_clusters(zone)`, que ejecuta `DBSCAN` (`ST_ClusterDBSCAN`) sobre los reportes activos de una zona.

Las migraciones consisten en 6 archivos SQL en `supabase/migrations/`, ejecutados en orden: extensiones (PostGIS), tablas, RLS, funciones RPC, vistas y datos iniciales. Los microservicios nunca se conectan directo a PostgreSQL; usan `supabase-py` (PostgREST/RPC) con la `service_role` key, lo que evita agotar el pool de conexiones en Lambda.

## 10. Seguridad y Gestion de Secretos

Esta terminantemente prohibido incluir archivos `.env`, llaves de API, credenciales de base de datos o secretos en los repositorios de codigo, ni siquiera en commits historicos. Los microservicios en Lambda recuperan sus configuraciones y secretos en tiempo de inicializacion directamente desde AWS Secrets Manager.

El flujo de recuperacion de secretos funciona de la siguiente manera: en `libs/emergencias_common/supabase_client.py`, la funcion `get_supabase_client()` primero intenta leer `SUPABASE_URL` y `SUPABASE_SERVICE_ROLE_KEY` de variables de entorno. Si no existen y detecta que esta en Lambda (`AWS_LAMBDA_FUNCTION_NAME` existe), llama a `_load_secrets_from_aws()`, que usa `boto3` para leer el secreto `emergencias/supabase` de Secrets Manager, parsea el JSON y devuelve URL y `service_role_key`. El resultado se cachea con `@lru_cache` para que solo se consulte una vez por cold start.

Row-Level Security (RLS) garantiza que cada operador solo acceda a datos de su zona. Las politicas SQL filtran por `zone_id` basado en el `auth.uid()` del JWT. Los ciudadanos anonimos nunca interactuan con Supabase directamente; todo pasa por API Gateway hacia S1, que usa `service_role` key. En Vercel, solo se inyectan variables publicas de cliente (`anon key`) a traves del panel de configuracion seguro, sin exponer la Service Role Key.

## 11. Infraestructura AWS

| Servicio | Uso                                                                                                                                                               |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Lambda   | 4 funciones: <code>emergencias-s1-intake</code> , <code>s2-dispatch</code> , <code>s3-geospatial</code> , <code>s4-notification</code> . Imagen Docker desde ECR. |
| ECR      | 4 repositorios de imagenes Docker. Build con <code>--platform linux/amd64</code> .                                                                                |

|                 |                                                                                          |
|-----------------|------------------------------------------------------------------------------------------|
| API Gateway     | HTTP API que rutea /reports a S1 y /dispatch a S2. Configuración de CORS.                |
| Secrets Manager | Secreto emergencias/supabase con SUPABASE_URL y SUPABASE_SERVICE_ROLE_KEY.               |
| IAM             | Rol emergencias-lambda-role con permisos para CloudWatch, Secrets Manager y EventBridge. |

*Tabla 2. Servicios AWS utilizados en producción.*

Las funciones usan imagen Docker base public.ecr.aws/lambda/python:3.12. Las imágenes se construyen con `docker buildx build --platform linux/amd64 --provenance=false --output type=docker`, ya que Lambda solo acepta amd64 y no soporta el formato OCI con provenance metadata.

## 12. Canary Deployment

El despliegue utiliza la estrategia de Canary Releases con Lambda aliases y weighted routing para minimizar el riesgo en producción. El proceso es el siguiente: (a) se actualiza el código de la Lambda con la nueva imagen de ECR; (b) se publica una nueva versión numerada inmutable; (c) se actualiza el alias live para apuntar a la nueva versión con peso del 10 por ciento, mientras el 90 por ciento del tráfico sigue yendo a la versión anterior mediante `AdditionalVersionWeights`; (d) se esperan 30 segundos como ventana de observación; (e) si todo funciona, se promueve al 100 por ciento limpiando los `AdditionalVersionWeights`.

API Gateway apunta a las funciones mediante el alias live (por ejemplo, `emergencias-s1-intake:live`), no a `$LATEST`. Esto permite redirigir el tráfico instantáneamente sin modificar la configuración del Gateway. En caso de fallo, el rollback consiste en cambiar el alias de vuelta a la versión anterior.

## 13. CI/CD Pipeline

El pipeline de integración y despliegue continuo está implementado con GitHub Actions en el archivo `.github/workflows/deploy.yml`. Se ejecuta en cada push a la rama main y consta de tres jobs:

**Job 1 - Tests:** instala Python 3.12 y las dependencias, luego ejecuta `pytest` para los cuatro microservicios. Si algún test falla, el pipeline se detiene.



**Job 2 - Build Frontend:** se ejecuta en paralelo con los tests. Usa Node.js 20, ejecuta npm ci y npm run build para verificar que el frontend compila sin errores.

**Job 3 - Deploy:** solo se ejecuta en push a main (no en pull requests) y depende de que los dos jobs anteriores hayan pasado. Hace login a ECR, construye y sube las cuatro imagenes Docker, actualiza las cuatro funciones Lambda y ejecuta el canary deployment (10 por ciento de trafico a la nueva version, espera 30 segundos, promocion a 100 por ciento).

Los secretos de AWS (AWS\_ACCESS\_KEY\_ID y AWS\_SECRET\_ACCESS\_KEY) estan configurados en GitHub Settings, Secrets and variables, Actions. Nunca en el codigo fuente.

## 14. Gobernanza de Costos (AWS Budgets)

Se configuro un presupuesto mensual en AWS Budgets con nombre emergencias-mvp-monthly y un limite maximo de \$10.00 USD. Se establecieron dos alertas automaticas por correo electronico: la primera se activa cuando el consumo real supera el 50 por ciento del presupuesto (\$5.00); la segunda se activa cuando el consumo proyectado (forecasted) supera el 85 por ciento del presupuesto (\$8.50). La captura de pantalla de la configuracion se encuentra en docs/budget-overview.png del repositorio.

## 15. Diagramas C4

Se generaron tres niveles de diagramas C4 en formato SVG, almacenados en la carpeta docs/ del repositorio:

**Nivel 1 - Contexto (c4-nivel1-contexto.svg):** muestra el sistema como una caja negra con sus actores externos: Ciudadano, Operador, Supabase y Organismo Externo.

**Nivel 2 - Contenedores (c4-nivel2-contenedores.svg):** abre el sistema y muestra los componentes internos: Frontend PWA, API Gateway, 4 Lambdas, EventBridge, Secrets Manager, Supabase y Organismo Externo.

**Nivel 3 - Componentes (c4-nivel3-componentes.svg):** detalla los modulos internos del microservicio S1 Intake: Mangum Handler, FastAPI Router, Payload Validator (Pydantic), Priority Calculator, Zone Resolver, Event Publisher y Report Repository.

## 16. URLs de Produccion

| Recurso            | URL                                                                                                                         |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Frontend (Vercel)  | <a href="https://emergencias-mvp.vercel.app">https://emergencias-mvp.vercel.app</a>                                         |
| API Gateway (AWS)  | <a href="https://k5znqecid0.execute-api.us-east-2.amazonaws.com">https://k5znqecid0.execute-api.us-east-2.amazonaws.com</a> |
| Repositorio GitHub | <a href="https://github.com/Santiago1301/EmergenciasMvp">https://github.com/Santiago1301/EmergenciasMvp</a>                 |
| Video Demo         | <a href="https://youtu.be/6s8stPuHpzg">https://youtu.be/6s8stPuHpzg</a>                                                     |
| Region AWS         | us-east-2 (Ohio)                                                                                                            |

*Tabla 3. URLs de produccion del sistema.*

## 17. Referencias

Amazon Web Services. (2026). AWS Lambda Developer Guide.

<https://docs.aws.amazon.com/lambda/>

Brown, S. (2023). The C4 model for visualising software architecture. <https://c4model.com/>

Ester, M., Kriegel, H.-P., Sander, J. y Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining, 226-231.

FastAPI. (2026). FastAPI Documentation. <https://fastapi.tiangolo.com/>

Newman, S. (2021). Building Microservices: Designing Fine-Grained Systems (2a ed.). O'Reilly Media.

PostGIS. (2026). PostGIS Documentation. <https://postgis.net/documentation/>

Richardson, C. (2018). Microservices Patterns: With Examples in Java. Manning Publications.

Supabase. (2026). Supabase Documentation. <https://supabase.com/docs>